Langage C avancé — pointeurs et fichiers

Informatique — Licence 1 (L1)

Cours INF-L1-03

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Maîtriser la mémoire en C : pointeurs, allocation dynamique, fichiers et listes chaînées — les outils qui transforment un programmeur débutant en développeur capable de construire des applications complètes.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Les pointeurs : adresses mémoire et déréférencement
• Leçon 2 — Pointeurs et tableaux, arithmétique des pointeurs
• Leçon 3 — Allocation dynamique : malloc, calloc et free
• Leçon 4 — Les fichiers en C : lecture et écriture
• Leçon 5 — Listes chaînées et structures de données dynamiques
• Leçon 6 — Gestion des erreurs et techniques de débogage
• Leçon 7 — Projet : mini-application de gestion écrite en C
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Le cours précédent t'a appris le C « de surface » : variables, boucles, fonctions, tableaux. Ce cours descend dans la machine. En C, chaque variable vit à une adresse précise de la mémoire, et le langage te donne le pouvoir d'y accéder directement grâce aux pointeurs. C'est cette puissance qui rend le C indispensable pour les systèmes, les pilotes et les programmes rapides — et c'est elle qui explique aussi ses plantages les plus spectaculaires quand on la manie sans précaution.

Les pointeurs débloquent trois capacités décisives : modifier une variable depuis une fonction (tu as déjà croisé ce mécanisme sans le savoir : c'est le & de scanf), créer des données dont la taille n'est connue qu'à l'exécution (allocation dynamique), et construire des structures qui grandissent et rétrécissent selon les besoins, comme les listes chaînées. Les fichiers, eux, donnent une mémoire permanente à tes programmes : sans eux, toutes les données disparaissent à la fin de l'exécution.

La méthode de ce cours : chaque notion est d'abord dessinée (adresse, flèche, case mémoire), puis codée, puis piégée volontairement pour que tu reconnaisses l'erreur le jour où tu la commettras — car tu la commettras, comme tous les programmeurs C. L'objectif n'est pas d'éviter les plantages par peur, mais de savoir les comprendre et les corriger méthodiquement.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

• Expliquer ce qu'est une adresse mémoire et manipuler les opérateurs & et * sans les confondre.

• Utiliser les pointeurs pour modifier des variables depuis une fonction (passage par adresse).

• Exploiter le lien entre pointeurs et tableaux, et l'arithmétique des pointeurs.

• Allouer et libérer proprement de la mémoire avec malloc, calloc et free, en testant toujours le retour.

• Lire et écrire des fichiers texte en C (fopen, fprintf, fscanf, fgets, fclose).

• Construire et parcourir une liste chaînée simple (création, ajout en tête, affichage, libération).

• Diagnostiquer les erreurs classiques (segfault, fuite mémoire, pointeur pendant) avec une méthode de débogage.

Leçon 1 — Les pointeurs : adresses mémoire et déréférencement

La mémoire de l'ordinateur est un immense alignement de cases numérotées : chaque numéro est une adresse, et chaque variable occupe une ou plusieurs cases selon son type. L'opérateur & (« adresse de ») donne l'adresse d'une variable : si int age = 20 est rangé à l'adresse 0x1000, alors &age vaut 0x1000. Un pointeur est simplement une variable dont le contenu est une adresse : int *p; déclare p comme « pointeur sur int », capable de contenir l'adresse d'un entier. Après p = &age, on dit que p pointe sur age.

L'opérateur * (« contenu de »), appliqué à un pointeur, accède à la valeur rangée à l'adresse pointée : *p vaut 20, et l'affectation *p = 21 modifie directement age. C'est le déréférencement. Toute la difficulté du sujet tient à ce double rôle du symbole * : dans une déclaration, il annonce un pointeur (int *p) ; dans une expression, il suit le pointeur jusqu'à la valeur (*p). Deux phrases à retenir : p est une adresse, *p est la valeur à cette adresse.

Un pointeur qui ne pointe nulle part doit contenir NULL, la valeur conventionnelle « aucune adresse ». Déréférencer NULL ou un pointeur jamais initialisé (qui contient une adresse au hasard) provoque le fameux Segmentation fault : le système arrête brutalement le programme qui a touché une mémoire interdite. La discipline est donc absolue : tout pointeur naît soit avec une vraie adresse, soit avec NULL — jamais avec du hasard.

L'essentiel
&variable = son adresse ; un pointeur stocke une adresse ; *pointeur = la valeur à cette adresse (modifiable). Un pointeur sans cible vaut NULL, et on ne déréférence jamais NULL ni un pointeur non initialisé.

Exemple — Première manipulation d'un pointeur

```c
#include <stdio.h>

int main(void)
{
int age = 20;
int *p = &age; // p contient l'adresse de age

printf("age = %d\n", age); // 20
printf("adresse = %p\n", (void*)p);
printf("via p : %d\n", *p); // 20

*p = 21; // modifie age a travers p
printf("age = %d\n", age); // 21
return 0;
}
```

À retenir

- p est l'adresse, *p est la valeur pointée : deux objets différents, à ne jamais confondre.
- Le type du pointeur (int*, double*…) doit correspondre au type de la variable pointée.
- Initialiser tout pointeur à une adresse valide ou à NULL dès sa déclaration.

Exercice

Après le code suivant, que valent x et y ? int x = 5, y = 10; int *p = &x; *p = y + 2; p = &y; *p = *p + x; — détaille le rôle de p à chaque ligne, et explique pourquoi « p = &y » ne copie pas la valeur de x dans y.

✔ Corrigé
Ligne 1 : p pointe sur x. Ligne 2 : *p = y + 2 écrit 12 dans la case pointée, donc x = 12 (y reste 10). Ligne 3 : p change de cible et pointe maintenant sur y — changer de cible ne copie rien : x garde 12, y garde 10. Ligne 4 : *p = *p + x écrit dans y : 10 + 12 = 22. Résultat final : x = 12, y = 22. Retenir : affecter un pointeur (p = …) déplace la flèche ; affecter *p modifie la case au bout de la flèche. Toute la matière tient dans cette distinction.

Leçon 2 — Pointeurs et tableaux, arithmétique des pointeurs

En C, le nom d'un tableau désigne l'adresse de sa première case : int t[5]; fait que t vaut &t[0]. C'est pour cela que scanf lit une chaîne sans & : le nom est déjà une adresse. Conséquence immédiate : t[i] et *(t + i) sont deux écritures du même accès. Le pointeur et le tableau sont deux vues du même alignement de cases en mémoire — les éléments d'un tableau sont toujours rangés côte à côte, dans l'ordre.

L'arithmétique des pointeurs exploite cette contiguïté : si p pointe sur un int, p + 1 désigne l'adresse de l'int suivant, c'est-à-dire 4 octets plus loin (la taille d'un int) — et non « l'adresse plus 1 ». Le pointeur avance par pas de la taille du type pointé : pour

un double, p + 1 saute 8 octets. On peut donc parcourir un tableau avec un pointeur qui avance (p++) exactement comme avec un indice, et calculer des distances : p2 − p1 donne le nombre d'éléments entre deux positions du même tableau.

Ce lien explique aussi le passage des tableaux aux fonctions : quand tu écris void f(int t[]), la fonction reçoit en réalité un pointeur sur la première case — elle travaille donc sur le tableau ORIGINAL, contrairement aux variables simples copiées. C'est pourquoi une fonction peut trier ou remplir ton tableau, mais aussi pourquoi elle doit recevoir la taille en paramètre : un pointeur seul ne dit pas où le tableau se termine.

L'essentiel
Le nom d'un tableau = l'adresse de sa case 0 ; t[i] ≡ *(t + i). p + 1 avance d'une case du type pointé. Un tableau passé à une fonction devient un pointeur : la fonction agit sur l'original et a besoin de la taille.

Exemple — Parcourir un tableau par pointeur

```
#include <stdio.h>

int main(void)
{
int t[5] = {10, 20, 30, 40, 50};
int *p;

for (p = t; p < t + 5; p++) {
printf("%d ", *p);
}
printf("\n");
// Affiche : 10 20 30 40 50
// *(t + 2) vaut 30, comme t[2]
return 0;
}
```

À retenir

- t, &t[0] et un pointeur initialisé à t désignent le même point de départ.
- L'incrément d'un pointeur dépend du type : +1 case, pas +1 octet.
- Comparer deux pointeurs (p < t + 5) n'a de sens qu'au sein d'un même tableau.

Exercice

Écris une fonction int sommeTableau(int *t, int n) qui calcule la somme d'un tableau en le parcourant uniquement avec l'arithmétique des pointeurs (aucun crochet [] dans le corps). Puis explique pourquoi la fonction modifie potentiellement le tableau si elle écrivait *p = 0 par erreur, alors qu'une fonction recevant un int simple ne risque rien de tel.

✔ Corrigé
int sommeTableau(int *t, int n) { int s = 0; int *p; for (p = t; p < t + n; p++) s += *p; return s; } — le pointeur p avance case par case jusqu'à la borne t + n (adresse juste après la dernière case, licite à calculer mais jamais à déréférencer). Différence fondamentale : recevoir un pointeur, c'est recevoir la clé de la maison ; recevoir un int, c'est recevoir une photocopie d'un nombre. Une écriture par *p atteint la mémoire de l'appelant : puissance et danger dans le même geste.

Leçon 3 — Allocation dynamique : malloc, calloc et free

Un tableau déclaré int t[30] a une taille figée à l'écriture du programme. Mais combien d'étudiants s'inscriront cette année ? On ne le sait qu'à l'exécution. L'allocation dynamique permet de demander de la mémoire au moment où on en connaît la taille : malloc(n * sizeof(int)) réserve, dans la zone mémoire appelée le tas, la place pour n entiers et retourne l'adresse du bloc — ou NULL si la mémoire manque. sizeof donne la taille d'un type en octets, donc le calcul s'adapte à toutes les machines.

calloc fait le même travail en initialisant toutes les cases à zéro — pratique pour des compteurs ou des structures. Dans les deux cas, la règle d'or suit immédiatement l'appel : tester le retour (if (p == NULL) { erreur ; }) avant tout usage. Un malloc non testé transforme un manque de mémoire en segfault incompréhensible.

La mémoire du tas ne se libère pas toute seule : chaque malloc doit rencontrer son free(p), qui rend le bloc au système. Oublier free provoque une fuite mémoire — le programme grossit jusqu'à épuiser la machine, critique pour un programme qui tourne longtemps. Après un free, le pointeur devient « pendant » : il vise une zone rendue, qu'il ne faut plus jamais toucher ; la bonne habitude est de remettre p = NULL juste après le free. La discipline se résume en trois temps : allouer, vérifier, utiliser, libérer, annuler.

L'essentiel
malloc réserve sur le tas et retourne une adresse ou NULL ; calloc initialise en plus à zéro. Tester NULL, toujours. Chaque allocation se termine par un free, suivi de p = NULL.

Exemple — Tableau de notes de taille choisie à l'exécution

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
int n, i;
double *notes;

printf("Combien de notes ? ");
scanf("%d", &n);

notes = malloc(n * sizeof(double));
if (notes == NULL) {
printf("Memoire insuffisante\n");
return 1;
}
for (i = 0; i < n; i++) {
printf("Note %d : ", i + 1);
scanf("%lf", &notes[i]);
}
printf("Premiere note : %.2f\n", notes[0]);
free(notes);
notes = NULL;
return 0;
}
```

À retenir

- malloc/calalloc vivent dans <stdlib.h> ; la taille se calcule avec sizeof, jamais en dur.
- Le test « == NULL » fait partie de l'appel, pas une option.
- free libère, NULL protège : un pointeur libéré puis annulé ne peut plus nuire.

Exercice

Écris un programme qui demande n, alloue dynamiquement un tableau de n entiers, le remplit des valeurs 1 à n, affiche la somme, libère la mémoire. Puis liste les trois fautes du fragment suivant : int *p = malloc(10); p[10] = 5; free(p); *p = 3;

✔ Corrigé
Programme : allocation de n * sizeof(int), boucle de remplissage p[i] = i + 1, somme cumulée, affichage, free(p), p = NULL. Les trois fautes : 1) malloc(10) réserve 10 OCTETS (2 entiers et demi), pas 10 entiers — il fallait 10 * sizeof(int) ; 2) p[10] serait déjà hors limites même avec la bonne allocation (indices 0 à 9) ; 3) *p = 3 après free écrit dans une mémoire rendue au système : usage d'un pointeur pendant, comportement indéfini. Ces trois fautes sont, dans l'ordre, les causes les plus fréquentes de plantages « aléatoires » en C.

Leçon 4 — Les fichiers en C : lecture et écriture

Sans fichiers, un programme perd tout à sa fermeture : les notes saisies, les fiches créées, les totaux calculés. Un fichier est une suite d'octets stockée durablement sur le

disque. En C, on manipule un fichier à travers un pointeur spécial, FILE*, obtenu par fopen(nom, mode) : le mode "w" écrit en écrasant, "a" ajoute à la fin, "r" lit un fichier existant. fopen retourne NULL si l'ouverture échoue (fichier absent en lecture, dossier interdit…) : encore un retour à tester systématiquement.

Les fonctions d'accès ressemblent à celles de l'écran et du clavier : fprintf(f, "…", …) écrit dans le fichier, fscanf lit des valeurs formatées, fgets lit une ligne entière (espaces compris) et fputs écrit une chaîne. La boucle de lecture classique lit jusqu'à ce que fscanf retourne EOF (fin de fichier) ou que fgets retourne NULL. Quand le travail est fini, fclose(f) ferme proprement le fichier et garantit que tout ce qui devait être écrit l'est réellement.

Le schéma professionnel, à retenir comme un réflexe : ouvrir, vérifier, traiter, fermer. La plupart des « mon fichier est vide » de débutant viennent d'un fclose oublié (les données restent dans un tampon en mémoire) ou d'un mode "w" qui a écrasé le fichier que l'on croyait compléter — pour conserver l'existant, c'est "a" qu'il faut choisir. Les fichiers binaires (fread/fwrite) stockent les données telles quelles en mémoire ; à ce niveau, les fichiers texte suffisent et ont l'avantage d'être lisibles dans n'importe quel éditeur.

L'essentiel
fopen(nom, mode) retourne un FILE* ou NULL : "r" lire, "w" écrire en écrasant, "a" ajouter. fprintf/fscanf/fgets écrivent et lisent ; fclose termine et sécurise les données.

Exemple — Enregistrer puis relire des noms d'étudiants

```
#include <stdio.h>

int main(void)
{
FILE *f = fopen("etudiants.txt", "w");
char ligne[100];

if (f == NULL) { printf("Ouverture impossible\n"); return 1; }
fprintf(f, "Awa\n");
fprintf(f, "Moussa\n");
fclose(f);

f = fopen("etudiants.txt", "r");
if (f == NULL) { printf("Lecture impossible\n"); return 1; }
while (fgets(ligne, 100, f) != NULL) {
printf("Lu : %s", ligne);
}
fclose(f);
return 0;
}
```

À retenir

- Tester fopen comme on teste malloc : NULL signifie échec, toujours.
- "w" détruit le contenu précédent ; "a" préserve et ajoute : le choix du mode est une décision, pas un détail.

- Sans fclose, rien ne garantit que les données sont écrites sur le disque.

Exercice

Écris un programme qui crée le fichier notes.txt contenant les nombres de 1 à 10, un par ligne, puis le relit et affiche leur somme (qui doit valoir 55). Indique le mode d'ouverture de chaque étape et ce qui se passerait si la deuxième ouverture utilisait "w" par erreur.

✔ Corrigé
Étape 1 : fopen("notes.txt", "w"), boucle fprintf(f, "%d\n", i) pour i de 1 à 10, fclose. Étape 2 : fopen en "r", boucle while (fscanf(f, "%d", &v) == 1) somme += v, fclose, afficher 55. Si la deuxième ouverture était en "w", le fichier serait vidé à l'ouverture avant toute lecture : la somme vaudrait 0 (aucune valeur lue) et le contenu serait perdu — l'erreur ne produit ni plantage ni message, juste un résultat silencieusement faux, la pire catégorie de bug.

Leçon 5 — Listes chaînées et structures de données dynamiques

Un tableau, même alloué dynamiquement, souffre de deux rigidités : sa taille est fixée à l'allocation, et insérer un élément au milieu oblige à décaler tous les suivants. La liste chaînée supprime ces contraintes : chaque élément (un nœud) vit dans son propre bloc alloué par malloc et contient, en plus de sa donnée, un pointeur vers le nœud suivant. Le dernier nœud pointe sur NULL. On accède à la liste par un seul pointeur, la tête, qui vise le premier nœud : perdre la tête, c'est perdre toute la liste.

Le nœud est une structure qui se désigne elle-même : typedef struct Noeud { int valeur; struct Noeud *suivant; } Noeud;. L'ajout en tête est le geste fondamental : créer un nœud par malloc, y mettre la valeur, lui donner comme suivant l'ancienne tête, puis faire de lui la nouvelle tête — trois lignes qui marchent aussi bien pour une liste vide (tête NULL) que pour une liste pleine. Le parcours suit les flèches : tant que le pointeur courant n'est pas NULL, traiter le nœud puis avancer par courant = courant->suivant. L'opérateur -> accède aux champs à travers un pointeur de structure.

Contrepartie honnête : pas d'accès direct à la case numéro 50 (il faut traverser les 49 précédentes), et chaque nœud coûte la taille d'un pointeur en plus. La liste chaînée excelle là où les insertions et suppressions sont fréquentes et la taille imprévisible ; le tableau gagne quand on accède partout par indice. Choisir sa structure selon l'usage, et non par habitude, est une compétence d'ingénieur que ce premier contact commence à former.

L'essentiel
Un nœud = une donnée + un pointeur vers le suivant ; la tête est la seule porte d'entrée. Ajout en tête en 3 temps (malloc, lier, devenir tête) ; parcours par flèches jusqu'à NULL.

Exemple — Construire et afficher une liste de trois nombres

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Noeud {
int valeur;
struct Noeud *suivant;
} Noeud;

Noeud* ajouterEnTete(Noeud *tete, int v)
{
Noeud *n = malloc(sizeof(Noeud));
if (n == NULL) return tete; // echec : liste inchangee
n->valeur = v;
n->suivant = tete;
return n;
}

int main(void)
{
Noeud *liste = NULL, *courant;
liste = ajouterEnTete(liste, 30);
liste = ajouterEnTete(liste, 20);
liste = ajouterEnTete(liste, 10);

for (courant = liste; courant != NULL; courant = courant->suivant)
printf("%d -> ", courant->valeur);
printf("NULL\n"); // Affiche : 10 -> 20 -> 30 -> NULL
return 0;
}
```

À retenir

- La liste se parcourt dans un seul sens, de la tête vers NULL ; tout traitement suit ce chemin.
- Libérer une liste = parcourir en sauvegardant le suivant AVANT de free chaque nœud.
- Ajouter en tête inverse l'ordre d'arrivée : le dernier ajouté est le premier affiché.

Exercice

Écris la fonction int longueur(Noeud *tete) qui retourne le nombre de nœuds d'une liste, sans la modifier. Puis décris précisément l'ordre des opérations pour libérer entièrement une liste de deux nœuds A -> B -> NULL, et ce qui se perd si l'on free la tête avant de sauvegarder son suivant.

✔ Corrigé
longueur : compteur à 0, pointeur courant part de la tête, tant que courant != NULL : compteur++, courant = courant->suivant ; retourner le compteur — la tête elle-même n'a jamais bougé, la liste est intacte. Libération : sauver B (suivant de A) dans un temporaire, free(A), passer à B, sauver son suivant (NULL), free(B). Si l'on free(A) d'abord sans sauvegarde, l'adresse de B — qui n'existait que dans le champ suivant de A — est perdue avec le bloc libéré : B devient inaccessible et sa mémoire fuit définitivement. Règle d'or : toujours sauvegarder la suite avant de détruire le maillon.

Leçon 6 — Gestion des erreurs et techniques de débogage

Un programme C qui « plante » ne donne souvent qu'un message laconique : Segmentation fault. Ce message signifie que le programme a lu ou écrit une adresse interdite. Les causes habituelles forment une courte liste à passer en revue dans l'ordre : pointeur non initialisé ou NULL déréférencé, indice de tableau hors limites, écriture par un pointeur après free, & oublié dans scanf (l'adresse reçue est alors une valeur quelconque), et retour d'une fonction pointant sur une variable locale déjà disparue.

La première technique de débogage ne demande aucun outil : la trace. On insère des printf numérotés aux endroits stratégiques (« étape 3, i = %d, p = %p ») pour découvrir la dernière étape franchie avant le plantage — le bug vit entre ce point et le suivant. En réduisant progressivement l'intervalle, on encercle la ligne coupable. La seconde technique est la réduction : commenter des morceaux jusqu'à obtenir le plus petit programme qui plante encore ; neuf fois sur dix, le bug devient alors évident.

Les outils professionnels prolongent ces réflexes : le débogueur (gdb) exécute pas à pas en montrant les variables ; les options de compilation -Wall -Wextra font parler le compilateur, dont les avertissements (warnings) sont des bugs annoncés à l'avance — un programme qui compile avec des warnings « bénins » est un programme en sursis ; -g ajoute les informations de débogage ; et valgrind (sous Linux) traque fuites et accès invalides en exécution. Cultiver ces réflexes dès la L1 vaut plus que n'importe quelle astuce de langage.

L'essentiel
Segfault = adresse interdite touchée. Méthode : tracer par printf pour localiser, réduire le programme pour isoler. Compiler toujours avec -Wall -Wextra et traiter chaque warning comme un bug.

Exemple — Le même bug, avant et après correction

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
int *p = NULL;
// *p = 5; // BUG : dereferencer NULL -> segfault

p = malloc(sizeof(int));
if (p == NULL) return 1;
*p = 5; // OK : la cible existe
printf("%d\n", *p);
free(p);
p = NULL;
return 0;
}
```

À retenir

- Un warning du compilateur n'est pas une opinion : c'est un défaut détecté gratuitement.
- Déboguer = localiser d'abord (où ça casse), comprendre ensuite (pourquoi), corriger enfin.
- Les suspects habituels du segfault tiennent en cinq causes — les passer en revue systématiquement.

Exercice

Ce programme plante parfois sans message clair. Trouve ses deux dangers et corrige-les : int t[5]; int *p; for (i = 0; i <= 5; i++) t[i] = i; scanf("%d", p);

✔ Corrigé
Danger 1 : la boucle va jusqu'à i <= 5, donc écrit dans t[5] qui n'existe pas (indices 0 à 4) — débordement de tableau, mémoire voisine corrompue. Correction : i < 5. Danger 2 : scanf("%d", p) avec p non initialisé : scanf écrit la valeur saisie à l'adresse aléatoire contenue dans p — plantage ou corruption selon la chance. Correction : p doit pointer sur une vraie variable (int x; p = &x;) ou un bloc alloué, ou plus simplement scanf("%d", &x). « Parfois » est la signature des bugs mémoire : ils dépendent de ce que contient la mémoire ce jour-là, et c'est exactement pour cela qu'on les traque par la méthode, pas par la chance.

Leçon 7 — Projet : mini-application de gestion écrite en C

Il est temps d'assembler tout le cours dans un programme réel : un gestionnaire de notes d'étudiants. Cahier des charges : saisir un nombre quelconque d'étudiants (nom + note), les stocker dynamiquement, afficher la liste, calculer la moyenne de la classe, rechercher un étudiant par son nom, et sauvegarder le tout dans un fichier pour le retrouver au prochain lancement. Chaque exigence correspond à une leçon : structures pour la fiche, malloc pour le stockage, parcours pour les calculs, fichiers

pour la persistance.

La conception précède le code. On définit d'abord la structure Etudiant { char nom[30]; double note; } et le tableau dynamique qui la contiendra. Puis on découpe le travail en fonctions courtes : saisirEtudiants, afficher, moyenneClasse, rechercher, sauvegarder, charger. Le menu principal (une boucle + un switch) ne fait qu'orienter vers ces fonctions. Ce découpage n'est pas une cérémonie : c'est lui qui permet de tester chaque morceau séparément et de corriger sans tout casser.

La démarche de réalisation suit l'ordre de risque croissant : d'abord l'affichage d'une fiche codée en dur, puis la saisie d'un étudiant, puis le tableau dynamique, puis les calculs, et seulement à la fin la sauvegarde fichier. À chaque étape, le programme compile et fonctionne — on ne reste jamais longtemps avec un programme « presque fini » qui ne marche pas du tout. Cette discipline de construction par tranches testées est la compétence professionnelle centrale de ce projet, bien au-delà du C lui-même.

L'essentiel
Un vrai programme = des structures claires, des fonctions courtes par tâche, un menu qui oriente, et une construction par étapes testées. Les fichiers donnent la persistance ; malloc donne la taille libre.

Exemple — Le squelette du projet (structures et prototypes)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
char nom[30];
double note;
} Etudiant;

Etudiant* saisirEtudiants(int *n);
void afficher(Etudiant *t, int n);
double moyenneClasse(Etudiant *t, int n);
int rechercher(Etudiant *t, int n, char *nom);
void sauvegarder(Etudiant *t, int n, char *fichier);

/* main : menu (1 saisir, 2 afficher, 3 moyenne,
4 rechercher, 5 sauvegarder, 0 quitter) */
```

À retenir

- Concevoir avant de coder : les données (structures) puis les fonctions, le code ensuite.
- Construire par tranches qui fonctionnent : chaque étape se compile et se teste.
- Le menu n'est qu'un aiguillage ; la valeur du programme est dans ses fonctions testées.

Exercice

Écris le corps de la fonction rechercher : elle reçoit le tableau, sa taille et un nom, et retourne l'indice de l'étudiant trouvé ou −1. Précise la fonction de comparaison utilisée et pourquoi un simple == entre les noms ne fonctionnerait pas.

✔ Corrigé
int rechercher(Etudiant *t, int n, char *nom) { int i; for (i = 0; i < n; i++) if (strcmp(t[i].nom, nom) == 0) return i; return -1; } — strcmp de <string.h> compare les contenus caractère par caractère et vaut 0 en cas d'égalité. L'opérateur == comparerait les ADRESSES des deux tableaux de caractères : deux noms identiques rangés à des endroits différents seraient déclarés différents. C'est la même règle qu'à la leçon sur les chaînes du cours précédent : en C, un nom de tableau est une adresse, jamais le contenu lui-même.

Travaux dirigés (TD)

TD 1 — Échanger pour de vrai : le passage par adresse

La fonction void echanger(int a, int b) qui échange a et b ne sert à rien : elle échange des copies. 1) Démontre-le en écrivant le programme qui l'appelle sur x = 3 et y = 7 et constate que rien n'a changé. 2) Réécris-la en void echanger(int *a, int *b) et l'appel correspondant pour qu'elle fonctionne réellement. 3) Explique, en une phrase par symbole, le rôle de * dans la déclaration, dans le corps, et l'absence de * à l'appel.

✔ Corrigé du TD 1
1) Après l'appel par valeur, x vaut toujours 3 et y toujours 7 : la fonction a échangé ses copies locales, détruites au retour. 2) Version correcte : void echanger(int *a, int *b) { int tmp = *a; *a = *b; *b = tmp; } appelée par echanger(&x, &y) — après l'appel, x = 7 et y = 3. 3) Dans la déclaration, * annonce « paramètre pointeur » ; dans le corps, * suit l'adresse pour lire/écrire la valeur de l'appelant ; à l'appel, on passe les adresses avec & : echanger(&x, &y). C'est exactement le mécanisme de scanf, qui « modifie » tes variables depuis le début du cours — le mystère est levé.

TD 2 — Sauvegarde et relecture d'un carnet de notes

Écris un programme qui : demande n, alloue un tableau dynamique de n structures Etudiant (nom, note), saisit les données, les écrit dans le fichier classe.txt (une ligne par étudiant : nom puis note), ferme le fichier, libère la mémoire. Puis un second programme (ou une seconde fonction) relit le fichier et affiche la moyenne de la classe. Fichiers et mémoire doivent être gérés sans fuite.

✔ Corrigé du TD 2
Saisie : malloc(n * sizeof(Etudiant)) avec test NULL ; boucle de scanf (nom en %29s, note %lf). Écriture : fopen("classe.txt", "w") testé, boucle fprintf(f, "%s %.2f\n", t[i].nom, t[i].note), fclose, free. Relecture : fopen en "r", boucle while (fscanf(f, "%29s %lf", nom, &note) == 2) { somme += note; compte++; } puis moyenne = somme / compte si compte > 0. Points de contrôle du correcteur : les deux fopen testés, les deux fclose présents, le free après usage, et le nombre de valeurs lues par fscanf vérifié (== 2) pour s'arrêter proprement en fin de fichier.

Travaux pratiques (TP)

TP 1 — Traquer un segfault avec méthode

Le programme fourni par l'enseignant plante par Segmentation fault dans environ un cas sur deux. Il contient trois des fautes classiques du cours (au choix parmi : pointeur non initialisé, borne de boucle dépassée, & manquant, usage après free, retour non testé de malloc). 1) Reproduis le plantage et note les entrées qui le provoquent. 2) Insère des printf de trace pour localiser la zone. 3) Corrige les trois fautes une par une, en recompilant avec -Wall -Wextra après chaque correction, et consigne chaque warning qui t'a aidé. Livrable : le programme corrigé + le journal des trois fautes (symptôme, cause, correction).

✔ Attendus du TP 1
Méthode attendue plutôt que réponse unique : le journal doit relier chaque symptôme à sa cause racine — par exemple « plantage seulement quand n dépasse 10 » → tableau statique de 10 débordé par la saisie ; « adresses affichées différentes à chaque exécution » → pointeur non initialisé lu par printf. Les warnings typiques : « variable used uninitialized », « format %d expects int* » (le & manquant se VOIT dans les warnings de format de scanf quand on compile avec -Wall). Un TP réussi est un journal où chaque correction est justifiée par une observation, pas une liste de changements au hasard.

Exercices corrigés supplémentaires

Exercice 1

Après int a = 8; int *p = &a; int *q = p; *q = *p + 4; que valent a, *p et *q ?

✔ Corrigé
p et q contiennent la même adresse (celle de a) : ce sont deux flèches vers la même case. *q = *p + 4 écrit 8 + 4 = 12 dans a. Résultat : a = 12, *p = 12, *q = 12. Deux pointeurs peuvent partager une cible — modifier par l'un se voit par l'autre, ce qui est tantôt un outil (plusieurs accès au même objet) tantôt un piège (modification surprise).

Exercice 2

Écris le code qui alloue dynamiquement un tableau de 100 réels initialisés à zéro, vérifie l'allocation, puis libère proprement. Justifie le choix entre malloc et calloc pour ce cas précis.

✔ Corrigé
double *t = calloc(100, sizeof(double)); if (t == NULL) { /* erreur */ } … free(t); t = NULL; — calloc est le bon choix ici car l'initialisation à zéro est exigée : avec malloc, il faudrait une boucle de mise à zéro, plus longue et plus risquée à oublier. calloc prend deux arguments (nombre d'éléments, taille d'un élément) là où malloc n'en prend qu'un (la taille totale).

Exercice 3

Un fichier texte contient un entier par ligne. Écris la boucle qui les lit tous et s'arrête proprement à la fin du fichier, sans connaître leur nombre à l'avance.

✔ Corrigé
int v; while (fscanf(f, "%d", &v) == 1) { /* traiter v */ } — fscanf retourne le nombre de conversions réussies : 1 tant qu'un entier est lu, EOF (ou 0) à la fin ou sur une donnée malformée. Tester le retour == 1 est la condition d'arrêt propre ; la boucle populaire « while (!feof(f)) » est fautive car feof ne devient vrai qu'APRÈS une lecture ratée, ce qui traite la dernière valeur deux fois — un classique des copies d'examen.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Si int x = 10 et int *p = &x, alors *p vaut…
   A) L'adresse de x
   B) 10
   C) L'adresse de p
2. Un pointeur qui ne pointe sur rien doit contenir…
   A) 0 obligatoirement écrit 0
   B) NULL
   C) L'adresse de main
3. malloc(20 * sizeof(int)) réserve…
   A) 20 octets
   B) 20 entiers
   C) 20 pointeurs
4. Après free(p), la première chose à faire est…
   A) Utiliser *p une dernière fois
   B) Rien, p est automatiquement détruit
   C) Mettre p = NULL
5. t[i] équivaut à…
   A) *(t + i)
   B) &t + i
   C) t + *i
6. Le mode d'ouverture qui ajoute sans effacer est…
   A) "w"
   B) "r"
   C) "a"

7. Dans une liste chaînée, le dernier nœud se reconnaît car…

A) Sa valeur vaut 0

B) Son pointeur suivant vaut NULL

C) C'est la tête

8. L'en-tête à inclure pour malloc et free est…

A) <stdio.h>

B) <stdlib.h>

C) <string.h>

9. fscanf retourne EOF quand…

A) Il a lu une valeur

B) La fin du fichier est atteinte sans lecture possible

C) Le fichier est en mode "w"

10. Compiler avec -Wall -Wextra sert à…

A) Rendre le programme plus rapide

B) Afficher les avertissements qui révèlent des bugs probables

C) Créer le fichier exécutable en couleur

Corrigés du QCM

1. Réponse B — *p suit l'adresse et donne la valeur rangée dans x : 10.

2. Réponse B — NULL est la valeur conventionnelle « aucune adresse » ; la déréférencer provoque un segfault.

3. Réponse B — La taille demandée est 20 × la taille d'un int : de quoi ranger 20 entiers.

4. Réponse C — Le pointeur devient pendant ; le remettre à NULL rend tout usage accidentel détectable (segfault net plutôt que corruption).

5. Réponse A — Le nom du tableau est l'adresse de base ; décaler de i cases et déréférencer donne la case i.

6. Réponse C — "a" (append) positionne en fin de fichier ; "w" écrase, "r" lit seulement.

7. Réponse B — La convention NULL termine la chaîne ; les valeurs des nœuds n'ont aucun rôle de fin.

8. Réponse B — Les fonctions d'allocation vivent dans <stdlib.h>.

9. Réponse B — EOF signale qu'aucune donnée n'a pu être lue ; c'est la condition d'arrêt propre des boucles de lecture.

10. Réponse B — Les warnings signalent des constructions suspectes (le & oublié dans scanf, une variable non initialisée…) avant qu'elles ne plantent en production.

Résumé — les notions importantes

• Toute variable vit à une adresse ; & la révèle, un pointeur la stocke, * la suit pour lire ou modifier la valeur pointée.

• Le passage par adresse (pointeurs en paramètres) permet enfin aux fonctions de modifier les variables de l'appelant — c'est le secret de scanf.

• Nom de tableau = adresse de la première case ; arithmétique des pointeurs = avancer par cases du type pointé.

• malloc/calloc allouent sur le tas (tester NULL !), free libère, et p = NULL désamorce le pointeur pendant : allouer, vérifier, utiliser, libérer, annuler.

• fopen/fprintf/fscanf/fgets/fclose donnent la persistance ; le mode choisit entre lire, écraser ou ajouter.

• La liste chaînée (nœud = donnée + pointeur suivant, tête unique) grandit sans limite fixe, au prix d'un accès uniquement séquentiel.

• Face au segfault : tracer, réduire, passer en revue les cinq causes classiques — et compiler toujours avec -Wall -Wextra.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Les pointeurs : adresses mémoire et déréférencement
p est l'adresse, *p est la valeur pointée : deux objets différents, à ne jamais confondre. ; Le type du pointeur (int*, double*...) doit correspondre au type de la variable pointée. ; Initialiser tout pointeur à une adresse valide ou à NULL dès sa déclaration.

Fiche 2 — Pointeurs et tableaux, arithmétique des pointeurs
t, &t[0] et un pointeur initialisé à t désignent le même point de départ. ; L'incrément d'un pointeur dépend du type : +1 case, pas +1 octet. ; Comparer deux pointeurs (p < t + 5) n'a de sens qu'au sein d'un même tableau.

Fiche 3 — Allocation dynamique : malloc, calloc et free
malloc/calalloc vivent dans <stdlib.h> ; la taille se calcule avec sizeof, jamais en dur. ; Le test « == NULL » fait partie de l'appel, pas une option. ; free libère, NULL protège : un pointeur libéré puis annulé ne peut plus nuire.

Fiche 4 — Les fichiers en C : lecture et écriture
Tester fopen comme on teste malloc : NULL signifie échec, toujours. ; "w" détruit le contenu précédent ; "a" préserve et ajoute : le choix du mode est une décision, pas un détail. ; Sans fclose, rien ne garantit que les données sont écrites sur le disque.

Fiche 5 — Listes chaînées et structures de données dynamiques
La liste se parcourt dans un seul sens, de la tête vers NULL ; tout traitement suit ce chemin. ; Libérer une liste = parcourir en sauvegardant le suivant AVANT de free chaque nœud. ; Ajouter en tête inverse l'ordre d'arrivée : le dernier ajouté est le premier affiché.

Fiche 6 — Gestion des erreurs et techniques de débogage
Un warning du compilateur n'est pas une opinion : c'est un défaut détecté gratuitement. ; Déboguer = localiser d'abord (où ça casse), comprendre ensuite (pourquoi), corriger enfin. ; Les suspects habituels du segfault tiennent en cinq causes — les passer en revue systématiquement.

Fiche 7 — Projet : mini-application de gestion écrite en C
Concevoir avant de coder : les données (structures) puis les fonctions, le code ensuite. ; Construire par tranches qui fonctionnent : chaque étape se compile et se teste. ; Le menu n'est qu'un aiguillage ; la valeur du programme est dans ses fonctions testées.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Pointeurs | Langage C - EP13

Chaîne : EvoluNoob · Langue : Français · Niveau : Débutant à intermédiaire
L'épisode de référence sur les pointeurs : ce qui les différencie des variables classiques, déclaration et initialisation, schémas mémoire, manipulation de données à travers un pointeur, passage à une fonction et fonction qui renvoie un pointeur.

https://www.youtube.com/watch?v=K_FhgzTyBMQ

Leçon 1 — #25 Langage C -Les Pointeurs - Partie 1 (Définition)

Chaîne : Programming With HAIRIBI · Langue : Français · Niveau : Débutant
La définition d'un pointeur expliquée simplement, première partie d'une série en cinq volets (pointeurs et tableaux, structures, passage par adresse, allocation dynamique) : idéal pour démarrer la leçon 1 en douceur.

https://www.youtube.com/watch?v=QTheWic1knw

Leçon 2 — Langage C-8 : Pointeurs, passage par valeur, passage par adresse et arithmétique des pointeurs.

Chaîne : Hassan EL BAHI · Langue : Français · Niveau : Débutant à intermédiaire
Passage par valeur contre passage par adresse, relation entre pointeurs et tableaux et arithmétique des pointeurs, avec cinq exercices pratiques pour ancrer la leçon 2.

https://www.youtube.com/watch?v=adnyaD1bVR4

Leçon 4 — Fichiers | Langage C - EP20

Chaîne : EvoluNoob · Langue : Français · Niveau : Intermédiaire
Tout sur les fichiers en C : le type FILE, ouvrir et fermer, lire et écrire en mode texte puis en mode binaire, déplacer le curseur — avec des exemples tirés du noyau Linux pour voir le vrai code.

https://www.youtube.com/watch?v=lD60NnWPcRA

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Kernighan & Ritchie — « Le langage C » : les chapitres 5 (pointeurs et tableaux) et 7 (entrées/sorties et fichiers) sont la référence d'origine.

• cppreference — référence du langage C (fr.cppreference.com) : fiches malloc, calloc, free, fopen et des fonctions de fichiers.

• Documentation de gdb (sourceware.org/gdb) et de valgrind (valgrind.org) pour le débogage et la traque des erreurs mémoire.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.